

ARCHITEKTUR-REVIEW

RankingGame

Projektstand verstehen, Architekturprinzipien erkennen und gezielt weiterlernen

Analysiert auf Commit 6fe927f

Codebasis, Abhängigkeitsrichtungen, Tests, Persistenz, Realtime und Frontend

Kurzurteil

Ein bemerkenswert substanzielles Lern- und Portfolio-Projekt: spielbarer Multiplayer-Kern, persistenter Zustand, transaktional gekoppelte Realtime-Events und Tests auf mehreren Ebenen. Die Zielarchitektur ist klar erkennbar, wird aber noch nicht überall konsequent eingehalten. Der wichtigste nächste Schritt ist nicht mehr Technik, sondern schärfere Grenzen und ein vollständig geschlossener Spielzyklus.

1. Das Wichtigste in fünf Minuten

137	64	20	198
Backend-Tests erfolgreich	Frontend-Tests erfolgreich	Flyway-Migrationen validiert	Java-Produktionsdateien

Domain-Unabhängigkeit



Spring/JPA-frei, aber Jackson + Use-Case-Imports

Adapter-Trennung



Controller meist schlank; eine Entity-Leakage

Use-Case-Klarheit



Gute Benennung, aber stark Entity-orientiert

Realtime-Konsistenz



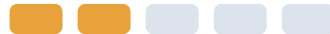
Events erst AFTER_COMMIT

Teststrategie



137 Backend + 64 Frontend erfolgreich

Feature-Vollständigkeit



Mehrere Runden und Scoring fehlen

Gesamturteil

Architektonisch solide mit klarer Lernkurve. Besonders stark sind Domain-Regeln, transaktionale Realtime-Publikation und Testbreite. Die größten Hebel liegen bei Grenzverletzungen, Feature-Abschluss und Frontend-Orchestrierung.

Was bereits trägt

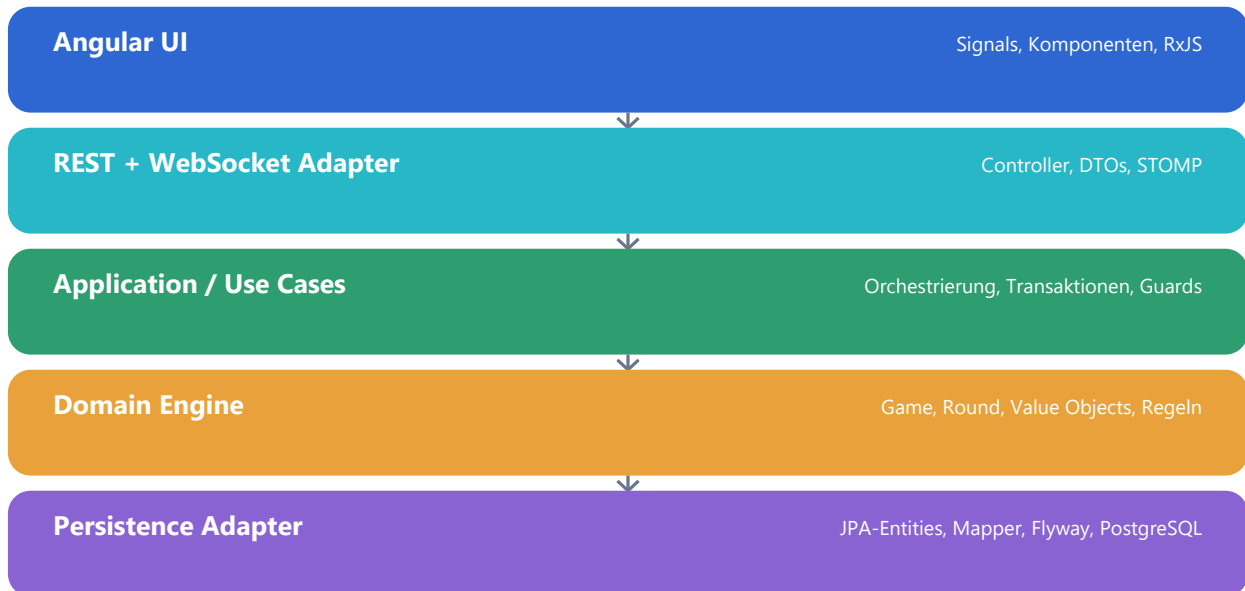
- Geschäftsregeln sind sichtbar: Game und Round modellieren Zustände, Berechtigungen und Übergänge.
- Realtime folgt Persistenz: WebSocket-Ereignisse werden erst nach erfolgreichem Commit veröffentlicht.
- Tests sind geschichtet: Domain, Use Cases, Controller, JPA, Integration, WebSocket und Angular.
- Der Stack ist produktionsnah: PostgreSQL, Flyway, sessionbasierte Spieleridentität und OpenAPI.

Was die Architektur aktuell bremst

- Der Domain-Kern kennt teilweise Use-Case-Typen und Jackson-Annotationen.
- Ein REST-Endpunkt gibt eine JPA-Entity direkt zurück.
- Application Services koordinieren Use Cases, arbeiten aber häufig direkt mit Persistenz-Entities.
- Der vollständige Zyklus aus mehreren Runden, Scoring und Spielende fehlt.
- Die Angular-Game-Komponente bündelt zu viel Laden, Zustand, Realtime und UI-Entscheidung.

2. Das System als Ganzes

Die intendierte Architektur ist eine pragmatische Schichtenarchitektur mit Elementen aus Clean Architecture und Domain-Driven Design. Der zentrale Gedanke: Je näher Code an der Fachlichkeit liegt, desto weniger soll er über Frameworks, Transport oder Datenbank wissen.



Abhängigkeitsregel

Äußere Schichten dürfen nach innen zeigen. Der Domain-Kern sollte nicht zurück auf Controller, Spring, JPA oder Use-Case-Pakete zeigen. Genau diese Richtung macht Regeln unabhängig testbar.

So liest du die Schichten

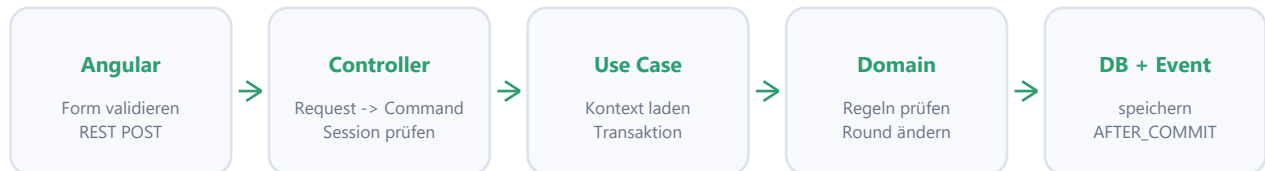
- Angular UI: zeigt Zustand an, sammelt Eingaben und reagiert auf REST/WebSocket.
- Adapter: übersetzen technische Protokolle in Anwendungsaufrufe und zurück.
- Application: beschreibt einen vollständigen Anwendungsfall und setzt Transaktionsgrenzen.
- Domain: schützt Invarianten - also Bedingungen, die immer wahr bleiben müssen.
- Persistence: speichert und rekonstruiert Zustand, ohne die Fachlogik zu definieren.

Lernprinzip: Architektur ist keine Ordnerstruktur

Ein Paketname allein schafft keine Grenze. Eine Grenze entsteht erst durch kontrollierte Imports, kleine öffentliche Verträge und Tests, die unerlaubte Abhängigkeiten sichtbar machen. Deshalb ist eine Importanalyse oft aussagekräftiger als ein schönes Diagramm.

3. Ein Use Case von außen nach innen

Die Antwortabgabe ist ein guter Querschnitt durch das System. Sie zeigt, wie HTTP, Autorisierung, Transaktion, Domain-Regeln, Persistenz und Realtime zusammenspielen.



Was dabei fachlich geschützt werden muss

- Die Runde gehört zum aktiven Spiel des Raums.
- Der Spieler gehört zum Spiel und darf genau einmal antworten.
- Antworten werden nur im Zustand ANSWER_SUBMISSION angenommen.
- Die letzte parallele Antwort darf den Übergang zu SORTING nur einmal auslösen.
- Clients dürfen das Sortier-Ereignis erst sehen, wenn der neue Zustand committed ist.

Starker Entwurf

Das pessimistische Locking in JpaRoundRepository schützt den nebenläufigkeitssensiblen Übergang. Das ist keine technische Nebensache: Parallelität ist Teil der Fachregel „genau einmal fortschreiten“.

Was du daraus lernen kannst

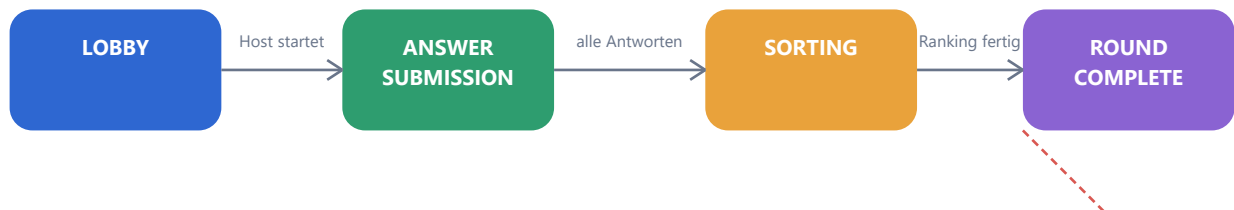
Ein Use Case ist mehr als eine Methode. Er ist eine atomare fachliche Geschichte: Eingabe akzeptieren, Berechtigung prüfen, Zustand verändern, speichern und die Außenwelt informieren. Wenn diese Geschichte über mehrere Schichten verteilt ist, sollte jede Schicht genau eine Art von Verantwortung behalten.

Prüffrage für neue Features

Kannst du den Ablauf als Satz formulieren - „Wenn X unter Bedingung Y passiert, wird Zustand Z atomar geändert und Ereignis E veröffentlicht“? Wenn nicht, ist der Use Case wahrscheinlich noch nicht klar genug geschnitten.

4. Domain-Modell: Regeln statt Datenbeutel

Der Kern mit Game, Round, Value Objects und fachlichen Exceptions ist die größte architektonische Stärke. Er macht Regeln explizit und direkt unit-testbar.



Noch nicht durchgezogen: Punkte auswerten -> nächste Runde -> Spielende -> Endstand

Gute Ansätze

- Value Objects wie AnswerText und IDs geben primitiven Werten Bedeutung und Validierung.
- Methoden mit Verhalten zentralisieren Zustandsänderungen statt Felder überall zu mutieren.
- Fachliche Exceptions dokumentieren, warum eine Aktion verboten ist.
- RoundStatus macht den Workflow als Zustandsmaschine sichtbar.

Die wichtigste Abweichung

Grenze ist noch durchlässig

Round.java importiert Typen aus usecases. Außerdem tragen mehrere Domain-IDs @JsonValue. Beides koppelt den Kern an äußere Belange. Es ist kein akuter Fehler, aber ein Signal: Die Dependency Rule wird nicht vollständig eingehalten.

Lernprinzip: Invarianten gehören zum Objekt, Orchestrierung zum Use Case

Die Runde selbst darf entscheiden, ob eine Antwort zulässig ist. Der Use Case entscheidet dagegen, welche Runde geladen, welcher Spieler authentifiziert und wann gespeichert wird. Diese Trennung verhindert sowohl „anämische“ Domain-Objekte als auch übergroße Aggregate, die plötzlich Datenbankzugriffe koordinieren.

5. Persistenz: Zwei Modelle, zwei Zwecke

Modell	Optimiert für	Sollte wissen über	Beispiel
Domain	Regeln und Invarianten	Spielbegriffe	Round, Game, AnswerText
JPA Entity	Relationale Speicherung	Tabellen, FK, Locks	RoundEntity, GameSession
API DTO	stabile Client-Verträge	JSON und Validierung	ActiveRoundResponse

Bewertung des aktuellen Stands

- Flyway besitzt die Hoheit über das Schema; Hibernate validiert. Das ist eine robuste Produktionspraxis.
- Mapper zeigen die bewusste Trennung zwischen Entity und Domain.
- Application Services und Repository-Interfaces sind allerdings stark Entity-orientiert. Dadurch ist die Persistenzgrenze eher organisatorisch als vollständig hexagonal.
- GameController.getActivePlayers gibt List<GameSessionPlayerEntity> zurück. Das verletzt die eigene Regel „keine JPA-Entities in REST-Responses“.

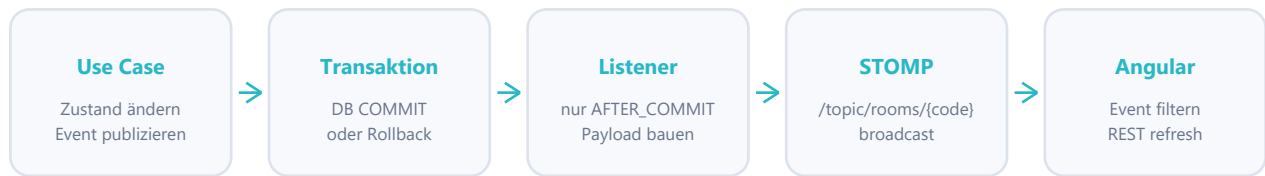
Konkreter nächster Schnitt

Für den Active-Players-Endpunkt ein Response-DTO verwenden. Danach schrittweise prüfen, ob Repository-Ports Domain-Objekte statt Entities liefern sollen. Nicht alles auf einmal umbauen: zuerst dort entkoppeln, wo ein API-Vertrag betroffen ist.

Lernprinzip: Mapping ist keine unnötige Doppelarbeit

Drei Modelle wirken zunächst redundant. In Wirklichkeit stabilisieren sie Veränderung: Eine Spaltenänderung muss nicht die API brechen, und eine fachliche Regel muss nicht von JPA-Lifecycle oder Lazy Loading abhängen. Der Preis sind Mapper; der Gewinn ist kontrollierte Kopplung.

6. Realtime: Ereignisse nach dem Commit



Warum AFTER_COMMIT so wichtig ist

Ohne diese Kopplung könnte ein Client ein Event empfangen, sofort per REST nachladen und noch den alten Datenbankzustand sehen. Durch `@TransactionalEventListener(phase = AFTER_COMMIT)` wird „Event sichtbar“ an „Zustand dauerhaft“ gekoppelt.

Was gut gelöst ist

- Events sind raumbezogen und werden über einen klaren Topic-Pfad verteilt.
- Payloads sind eigene Transporttypen statt JPA-Entities.
- Das Frontend behandelt Realtime vor allem als Änderungsimpuls und lädt kanonischen Zustand per REST nach.
- WebSocket-Verbindung und ausstehende Aktionen sind in einem eigenen Angular-Service gekapselt.

Risiken und Grenzen

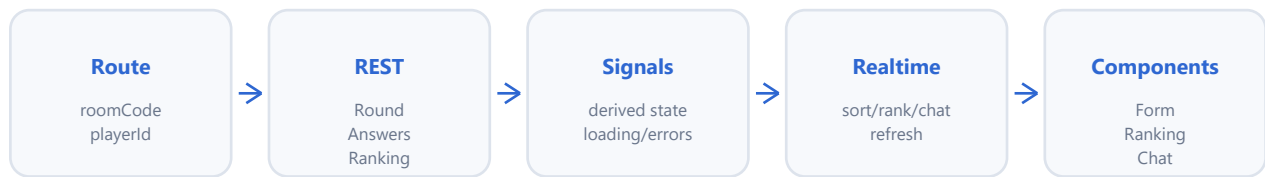
- AFTER_COMMIT garantiert keine Zustellung. Bei Prozessabsturz zwischen Commit und Send kann ein Event verloren gehen.
- Der einfache In-Memory-Broker und die lokale Presence-Registry skalieren nicht automatisch über mehrere Instanzen.
- Die Angular-Komponente enthält einen wachsenden Event-Dispatcher und manuelle Payload-Type-Guards.
- Fehler beim JSON-Parsen beenden aktuell den Observable-Stream des Subscribers.

Passende Evolutionsregel

Outbox, externer Broker oder verteilte Presence erst einführen, wenn Zuverlässigkeits- oder Skalierungsanforderungen es rechtfertigen. Für den aktuellen Projektumfang ist die einfache Lösung angemessen.

7. Frontend: moderne Reaktivität, wachsende Orchestrierung

Das Frontend nutzt Standalone Components, Signals, computed state, RxJS und klar getrennte API-/WebSocket-Services. Das ist modern und gut testbar. Gleichzeitig ist game.ts zum Orchestrierungszentrum geworden.



Was gut ist

- Computed Signals leiten UI-Zustand deterministisch ab.
- Form-Validierung verhindert triviale ungültige Requests.
- API- und WebSocket-Zugriff sind als Services isoliert und separat getestet.
- Präsentationskomponenten für Frage, Antwort, Ranking und Score Cards sind bereits extrahiert.

Wo die Komponente zu viel weiß

- Sie lädt mehrere Ressourcen und koordiniert deren Reihenfolge.
- Sie verwaltet parallele Loading- und Error-Zustände.
- Sie interpretiert Eventtypen und prüft Payloadformen.
- Sie enthält Session-/Routenlogik, fachliche UI-Entscheidungen und Netzwerkreaktionen.

Empfohlener Lernschritt

Eine GameFacade oder ein kleiner Feature Store kann Serverzustand, Commands und Event-Reduktion bündeln. Komponenten lesen dann ViewModel-Signals und senden Intents wie submitAnswer oder rankAnswer.

Nicht überabstrahieren

Die Extraktion lohnt sich erst dort, wo Zustandskoordination mehrfach vorkommt oder Tests schwer lesbar werden. Ein generischer Store-Framework-Wechsel wäre derzeit mehr Komplexität als Nutzen; eine feature-spezifische Facade genügt.

8. Teststrategie: Verhalten auf mehreren Ebenen

Ebene	Zweck	Aktueller Nachweis	Lernwert
Domain Unit	Regeln ohne Infrastruktur	Game, Round, Answer, Points	schnell, präzise
Use Case	Orchestrierung und Guards	Create/Join/Start/Submit/Rank	Geschichten testen
Adapter	HTTP/STOMP-Verträge	MockMvc + WebSocket	Grenzen sichern
Persistence	Mappings und Queries	JPA + PostgreSQL	echte Semantik
Integration/E2E	kritische Flows	Testcontainers + Playwright	Zusammenspiel

Verifiziert

mvn clean test: 137 Tests, 0 Fehler. Angular/Karma: 64 Tests, 0 Fehler.
Docker/Testcontainers und ChromeHeadless waren verfügbar. Playwright-E2E wurde nicht ausgeführt, weil dafür ein separat laufendes Backend erforderlich ist.

Ein aufschlussreicher Build-Befund

Ein erster inkrementeller mvn test-Lauf scheiterte durch veraltete Klassen im bestehenden target/. Nach mvn clean test war der Build grün. Das zeigt: Build-Ausgaben sind Cache, nicht Wahrheit. CI sollte immer aus einem sauberen Checkout bauen; lokal hilft ein Clean-Build, bevor man einen Fehler dem Quellcode zuschreibt.

Was als Nächstes getestet werden sollte

- Architekturtests für verbotene Imports und Entity-Leakage, z. B. mit ArchUnit.
- Mehr Runden, Scoring, Captain-Wechsel und Spielende als Domain- und Integrationsflow.
- Reconnect während ANSWER_SUBMISSION und SORTING.
- Ungültige oder unbekannte WebSocket-Payloads, ohne den Event-Stream dauerhaft zu beenden.
- Ein kleiner, stabiler End-to-End-Happy-Path in CI.

9. Findings nach Priorität

Prio	Befund	Warum relevant	Empfehlung
P1	Spielzyklus endet nach Ranking nicht vollständig	Kernversprechen des Spiels bleibt offen	Scoring, nächste Runde, Abschluss als vertikalen Slice bauen
P1	JPA-Entity im REST-Response	API wird an Persistenzschema gekoppelt	GameSessionPlayerResponse verwenden
P2	Domain importiert Use-Case-Typen	Abhängigkeitsrichtung wird umgedreht	fachliche Typen in engine verschieben
P2	Use Cases arbeiten breit mit Entities	Infrastrukturwechsel und Regeltests werden schwerer	Ports selektiv domain-orientieren
P2	Game-Komponente orchestriert zu viel	Änderungen erzeugen Seiteneffekte	GameFacade + Event-Reducer extrahieren
P3	Domain-IDs kennen Jackson	kleine Transportkopplung im Kern	Custom Serializer oder DTO-Mapping prüfen
P3	Realtime ohne durable delivery	Eventverlust bei Prozessabsturz möglich	erst bei echter Anforderung Outbox einführen

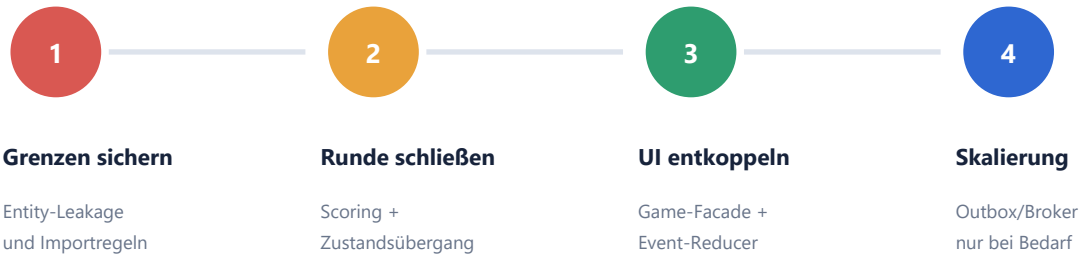
Wie Prioritäten zu lesen sind

P1 verbessert Produktvollständigkeit oder verhindert eine harte Grenzverletzung. P2 senkt die Änderungskosten der nächsten Features. P3 ist sinnvoll, sollte aber nicht vor realem Bedarf oder den P1/P2-Themen umgesetzt werden.

Bewusste Reihenfolge

Erst einen vollständigen Spielzyklus liefern und dabei die betroffenen Grenzen sauber ziehen. Eine großflächige „Clean-Architecture-Rewrite“ wäre riskanter und lehrärmer als mehrere kleine vertikale Schnitte.

10. Empfohlene Evolutionsroute



Phase 1 - Grenzen messbar machen

- Response-DTO für aktive Spieler einführen.
- ArchUnit-Regeln: engine importiert weder Spring/JPA/controller/usecases; Controller exportieren keine Entities.
- Package-Verantwortlichkeiten kurz in ADR oder README dokumentieren.

Phase 2 - Kernspiel schließen

- Domain-Entscheidung für Ranking-Auswertung und Punkteabzug modellieren.
- RoundComplete -> NextRound oder GameFinished als explizite Übergänge.
- Ein Integrationsflow über mindestens zwei Runden; UI zeigt Zwischen- und Endstand.

Phase 3 - Frontend entlasten

- GameFacade hält Remote State, Commands, Loading/Error und Event-Reaktionen.
- Realtime-Events über einen typisierten Reducer verarbeiten.
- Komponenten bleiben präsentationsnah und erhalten ViewModels.

Phase 4 - nur bei Wachstum

- Outbox für durable Events, externer Broker für mehrere Instanzen.
- Verteilte Presence und Idempotency Keys für wiederholte Commands.
- Observability mit fachlichen Metriken: aktive Räume, Rundenübergänge, Reconnects.

11. Die Prinzipien hinter dem Entwurf

1. Hohe Kohäsion Code, der aus demselben Grund geändert wird, gehört zusammen. Round hält Regeln; ein Controller hält HTTP-Übersetzung.	2. Niedrige Kopplung Schichten kennen möglichst kleine Verträge. DTOs und Ports schützen vor Kettenreaktionen.
3. Dependency Inversion Der fachlich wichtige Kern definiert, was er braucht; technische Adapter erfüllen diese Bedürfnisse.	4. Invarianten Regeln wie „nur einmal antworten“ müssen an jedem Eintrittspunkt gelten, nicht nur in der UI.
5. Zustandsmaschinen Explizite Zustände und erlaubte Übergänge sind leichter zu verstehen und zu testen als verstreute Boolean-Flags.	6. Atomarität Fachlich zusammengehörende Änderungen gehören in eine Transaktion. Events erst nach Commit halten Außenwelt und DB konsistent.
7. Ports & Adapter REST, WebSocket und JPA sind austauschbare Außenseiten; Use Cases und Domain bleiben der stabile Kern.	8. Evolution statt Big Bang Architektur wird durch kleine, überprüfbare Schnitte besser - idealerweise entlang eines echten Features.

Die wichtigste Meta-Lektion

Gute Architektur reduziert nicht die Zahl der Dateien, sondern die Zahl der Dinge, die du gleichzeitig im Kopf halten musst. Wenn du eine Spielregel änderst und nur Domain-Test, Use Case und einen Adaptervertrag anfassen musst, funktioniert die Architektur.

Praktische Übung

Nimm als nächsten Lern-Slice „Runde auswerten“. Schreibe zuerst die Zustandsübergänge und Invarianten auf, dann Domain-Tests, dann den Use Case, dann Persistenz und UI. Beobachte bewusst, welche Schichten aus welchem Grund geändert werden.

12. Checkliste für jede neue Story

Fachlichkeit	• Welche Invariante wird eingeführt oder verändert? • Ist der Zustandsübergang explizit? • Liegt die Regel in Domain statt Controller/UI?
Application	• Was ist die atomare Use-Case-Grenze? • Welche Berechtigung gilt? • Welches Ereignis entsteht nach Erfolg?
Persistenz	• Braucht es Locking oder Idempotenz? • Bleiben Entities hinter der Adaptergrenze? • Ist eine Flyway-Migration rückwärtsverträglich?
API & Realtime	• Sind Request/Response eigene Verträge? • Wird erst nach Commit publiziert? • Kann der Client Events doppelt oder verspätet verarbeiten?
Frontend	• Ist Serverzustand von lokalem UI-Zustand getrennt? • Sind abgeleitete Werte computed statt manuell synchronisiert? • Bleibt die Komponente präsentationsnah?
Tests	• Domain-Regel als Unit-Test? • Use Case als Verhaltensgeschichte? • Nur kritische Integrations-/E2E-Flows?

Definition of Done

Eine Story ist architektonisch fertig, wenn das Verhalten verständlich benannt, die Invariante zentral geschützt, die Transaktion eindeutig, der Außenvertrag stabil und der passende Test grün ist.

13. Evidenz und Untersuchungsumfang

Analysiert wurden Repository-Struktur, Build-Dateien, README, zentrale Backend- und Frontend-Flows, Importbeziehungen, TODOs, Migrationen und Tests. Die Aussagen beziehen sich auf den lokalen Stand am 27.07.2026, Commit 6fe927f.

Zentrale Codebelege

- Domain: engine/Game.java, engine/Round.java, engine/AnswerText.java, engine/GamePoints.java
- Use Cases: SubmitAnswerService.java, StartRankingGameService.java, RoundProgressService.java, RankAnswerService.java
- Persistenz: JpaRoundRepository.java (PESSIMISTIC_WRITE), entities/*, mapper/*, db/migration/*
- REST: GameController.java:51 (Entity-Response), AnswerController.java, GlobalExceptionHandler.java
- Realtime: RoomRealtimeEventPublisher.java:23ff. (AFTER_COMMIT), WebSocketService.ts
- Frontend: game/game.ts:37ff. und handleRealtimeEvent ab Zeile 373
- Tests: src/test/java, frontend/src/app/**/*spec.ts, frontend/e2e/*spec.ts

Ausgeführte Verifikation

- `.\mvnw.cmd clean test` - erfolgreich, 137 Tests, PostgreSQL 16 via Testcontainers.
- `npm.cmd test --prefix frontend -- --watch=false --browsers=ChromeHeadless` - erfolgreich, 64 Tests.
- Playwright-E2E nicht ausgeführt; der konfigurierte Flow erwartet ein separat erreichbares Backend.

Grenzen dieses Reviews

- Keine Last-, Security- oder Penetrationstests.
- Keine Produktionsmetriken oder reales Mehrinstanz-Deployment ausgewertet.
- Keine vollständige zeilenweise Codeprüfung aller 198 Java-Dateien.
- Keine Änderungen am Produktcode; dieses Dokument ist Analyse und Lernhilfe.

Schlussbild

RankingGame ist nicht nur ein CRUD-Projekt mit WebSocket. Es enthält bereits die schwierigen Themen: Zustandsübergänge, Nebenläufigkeit, Persistenz, Realtime-Konsistenz und mehrere Testebenen. Der nächste Reifegrad entsteht durch konsequentere Grenzen und das Schließen des vollständigen fachlichen Zyklus.